Load Bounds: Node, Edge and Lane Accounting for a Rings Node under Free Identities

Ryan J. Kung

Abstract

A Rings identity is a key digest, so anyone can mint as many as they like. Every table a remote party can grow must therefore be accounted to something that is not free. We take the authenticated neighbour edge: a connection the node itself admitted, which holds one of its $\mathcal{R}$ connection slots. One accounting law covers every such table: each slot has a reserved floor, overflow shares a pool in fair claims, and the node-wide bound is the sum. An honest edge is then refused only past its slot's floor and its fair claim, and a free identity consumes at most the share of the slot that carries it. We apply the law to the receiver's replay streams (with an eviction horizon that cannot reopen replay), to the removal store, to received frames through a credit-reclaim protocol that makes the frame bound hard and independent of the connection count, and to reassembled messages, which are charged once at their first chunk so that no honest message is refused. We state the complexity obligations of the admission hot paths, and replace the reassembly barrier's control exemption by a barrier per message category, under which order is guaranteed per category only.

I. Problem

#907
: The receiver's replay table never evicts, and a new stream fails closed once the table is full. Streams are keyed by the origin account, which costs nothing, so 16 384 key pairs and one message each lock a node out of every new origin, permanently.

#1060, residual
: The removal store charges each sender a share, and senders are free keys, so 64 identities fill it.

#934
: Credit bounds the frames held per connection, not per node: about 1.3 GiB at the connection cap. Advertised credit cannot be taken back, so no hard node-wide bound exists today.

#932
: A reassembled message is re-charged at its full size on the inbound actor, where it cannot wait, and is dropped when that charge does not fit. A chunk that finds the reassembly budget full is dropped too.

#938
: Admission costs $O(n)$ per event and takes node-global locks per frame, exactly when the node is saturated.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created October 25, 2026. This document specifies the load bounds of a Rings node before 1.0: what it holds node-wide, per neighbour edge and per lane, what a free identity can consume, and what refusal and backpressure look like at each bound (rings-transport, module callback; rings-core, modules fair_admission, swarm::callback::inbound, chunk, message::replay and dht::chord::carrier_store). It is the law for issues #907, #927, #932, #934, #938 and #1094 of RingsNetwork/rings, and closes the Sybil residual of the removal-stability law [1].

#927
: The reassembly barrier exempts control, which reorders chunked control, and orders data across categories, which no protocol needs.

II. Model

Definition 1 (Principals).

- An origin o is an account DID, recovered from a signature. Anyone can create one, at no cost.
- An edge e is a handshake-authenticated connection the node admitted (EdgeRelation::Neighbour). The registry holds at most $\mathcal{R} = 2(F+s+1)$ connection records in any phase, where $F = 160$ is the number of finger slots and s the number of successors (328 at $s = 3$). So at most $\mathcal{R}$ edges are current at any instant. An edge identity is also a free key; what is bounded is the number of edges current at once.
- A slot $\kappa \in K = \{0, \ldots, \mathcal{R} - 1\}$ is one unit of the registry's capacity. An edge takes the cleanest free slot $\kappa(e)$ when it is admitted and frees it when it is retired, both in the transition that announces it, so the current edges hold distinct slots. A slot is not a topology reference: one edge may back several fingers and a successor, or none. Let $\mathrm{use}^t_{T,\kappa}$ be the items of table T (Def. 2) charged to κ that are still live at t: an item past its instant is not counted, since the next admission of T retires it. The floor use of κ at t is its worst table's, $w_t(\kappa) = \max_T \min(\mathrm{use}^t_{T,\kappa}, f_T)/f_T$, the part of a floor that an edge admitted into κ would inherit; the cleanest free slot is the one of least w_t, the least index among equals.
- A lane (e, ℓ) is one of the four credit-controlled channels of an edge.

- The local principal λ is the node itself: its own operations alone (its own writes and repairs, and out-of-band connection signaling it is handed).
- A dedicated principal $d \in D_T$ is a duty of the node that remote input drives but no slot introduces, with a share of its own in table T: the inbox principal μ, which introduces deliveries out of the node's own relay inbox (replay table only), and the custody principal χ, which introduces the hand-off of a range the node owns (removal store only).

Definition 2 (Table). A table T is a node-wide set of items that remote input can grow: replay streams, removal records, frames held, reassembly bytes, mailbox bytes. Every item $x \in T$ carries a principal $\pi(x) \in K \cup \{\lambda\} \cup D_T$: the slot of the edge that introduced it, the local principal, or a dedicated one, and it leaves T only by a transition of T's own law (consumption, retirement at a horizon, or an acknowledged move).

Assumption 1 (What a remote peer can send). A remote party can:

- mint origins without bound;
- hold up to $\mathcal{R}$ edges at once, and replace an edge by a fresh identity whenever connection admission accepts one;
- send, on each edge, anything the edge's credit, quotas and admission laws let through.

It cannot authenticate as an edge it does not hold. Sybil and eclipse resistance remain non-goals; this law bounds what a Sybil consumes, and does not prevent one.

Refusal and backpressure. At a bound, a transition takes one of three forms:

- backpressure: the transition waits for capacity in resource order, and nothing is lost;
- refusal: it fails with an error that names the bound and changes nothing, and the sender keeps its copy;
- drop: it is discarded and counted.

A drop is allowed only where the law names it (best-effort relay, the application inbox). It is never allowed for an honest frame or message at a capacity bound.

III. The accounting law

Law 1 (Slot accounting). Every table T has a floor f_T per slot, a pool $\mathcal{P}_T$, a local share L_T and a share S_d per dedicated principal $d \in D_T$, with node-wide bound

$$B_T(s) = \mathcal{R}(s)\, f_T + \mathcal{P}_T + L_T + \textstyle\sum_{d \in D_T} S_d,$$
$$\mathcal{R}(s) = 2(F + s + 1),$$

so a deployment that raises the successor count s (#774, #777) raises the floors it reserves, $2f_T$ per successor, and nothing else. Let use_κ be the number of items charged to slot κ, and $o_\kappa = \max(0, \mathrm{use}_\kappa - f_T)$ its pool items. The claimants are the slots with $o_\kappa > 0$ or a pending item over their floor; let $n \geq 1$ be their number. An item introduced by π is admitted iff:

- $\pi = \kappa$ and $\mathrm{use}_\kappa < f_T$ (the slot's floor); or
- the pool has room, $\sum o < \mathcal{P}_T$, and the claimant is within its fair claim, $o_\kappa < \lceil \mathcal{P}_T / n \rceil$; or
- $\pi = \lambda$ and $\mathrm{use}_\lambda < L_T$; or
- $\pi = d \in D_T$ and $\mathrm{use}_d < S_d$.

A transition that adds several items is admitted iff each of them is, in order, and is refused whole otherwise. A refused transition is refused or, where the law says so, it waits. An item stays charged to its slot until it leaves by T's own law, whoever holds the slot: a departure changes no charge, and an edge admitted into a slot inherits its usage. No item is ever dropped to make room: a claimant above its fair claim after n grew keeps what it holds and gains nothing until it falls below.

Theorem 1 (Isolation). Under Law 1:

1) $|T| \leq B_T$ always;
2) an item through slot κ with $\mathrm{use}_\kappa < f_T$ is never refused, however many origins, edges or slots any other party controls; in particular an edge keeps its floor for as long as it holds its slot, less what the slot held when it was admitted. The guarantee is per item: a transition adding k items through κ is admitted only if $k \leq (f_T - \mathrm{use}_\kappa) + (\lceil \mathcal{P}_T / n \rceil - o_\kappa)$ with the pool's room, and is refused whole otherwise;
3) an over-floor slot is refused only once it holds its fair claim or the pool is full, and it is guaranteed $f_T + \lceil \mathcal{P}_T / n \rceil \geq f_T + \lceil \mathcal{P}_T / \mathcal{R} \rceil$ items as the other claimants' items expire, with $n - 1$ of them;
4) the items of the slots a party holds or has held number at most $j f_T + \mathcal{P}_T$ for j such slots, and at most $\mathcal{R} f_T + \mathcal{P}_T$ however many edges it rotates.

Proof. (1) No transition moves an item between slots or into the pool: a departure changes no charge. Each slot holds at most f_T items within its floor, the pool admits only while $\sum o < \mathcal{P}_T$, the local principal holds at most L_T, and each dedicated principal at most S_d. (2) The floor test reads use_κ alone, and only transitions through κ change it. (3) A claimant below $\lceil \mathcal{P}_T / n \rceil$ is refused only with a full pool, which then holds some claimant above its claim; that claimant gains nothing and its items leave by T's law, so the pool drains to the fair claims, with $n \leq \mathcal{R}$. (4) By (1), with $j \leq \mathcal{R}$. □

Proposition 1 (Rotation residual). A party that rotates edges through free slots charges each slot it passes with up to f_T items, which stay until they leave by T's law. An edge admitted at t inherits $\min_{\kappa \in F_t} w_t(\kappa)$ of its floors, over the free slots F_t, $|F_t| = \mathcal{R} - c$ with c edges current (Def. 1). So:

1) Targeted. The party takes any of a newcomer's floor only if it has charged every free slot with items live at the newcomer's admission: $\mathcal{R} - c$ admissions within the life h of those items, $h \leq H + 2\sigma$, or $H + H_I + 3\sigma$ for application streams.
2) Concurrent. A party that holds $\mathcal{R} - c$ edges at once, fills their floors and departs charges every free slot, under cleanest-first as under least-free, at the same

$\mathcal{R} - c$ admissions per horizon. Against it the choice of slot changes nothing.

3) Least-free. Had an edge taken the least free slot instead, (1) would fail: a departing edge usually frees the least free slot, which the next admission takes, so one admission per victim suffices.
4) An edge that holds its slot throughout is never affected.

Proof. Law 1: a slot's charges outlive its edge. (1) If some free slot has $w_t = 0$, the newcomer takes one, and $w_t(\kappa) > 0$ needs an item of κ live at t, charged by an edge admitted into κ within its life. (2) With every free slot charged, every choice inherits. (3) After the party's edge departs from the least free slot κ, κ is the least free slot unless a lesser one was freed meanwhile. (4) Theorem 1(2). □

Cleanest-first is a strict improvement and not a new bound. From any state, an admission inherits no more under it than under least-free, since it takes the least w_t over the same free slots; and it removes the targeted attack of (3), one admission per victim, leaving only the concurrent bound of (2). Proposition 1(2) is the cost this law does not remove: free keys and connection churn can leave newly admitted edges without a floor for one horizon of T per round of $\mathcal{R} - c$ admissions. Such a party is held only to its fair claim, $\lceil \mathcal{P}_T/n \rceil$, on the slots it charged, and an honest newcomer keeps that claim: degradation, never a refusal of an existing edge, and never a permanent lockout (Prop. 2).

Why not an admission rate limit (#1094). Keeping every free slot charged costs $(\mathcal{R} - c)/h$ admissions per unit time, about 3 a minute at $h = 100\,\text{min}$, $\mathcal{R} = 328$ and $c = 28$, and any positive rate at the application horizon. An honest node admits new edges at $c/\bar{L}$, with $\bar{L}$ the mean edge life: at most the mean session, 4.2 min for Weibull sessions of shape 0.5 and median 1 min with 90% browser nodes [2], so about 7 a minute for a browser with $c \approx 28$ and 70 for a seed with $c \approx 300$. A limit that never defers honest churn therefore admits more than the attack needs, and a limit tight enough to bound it, burst plus refill below $\mathcal{R} - c$ per horizon, would defer a seed's restart (about $\mathcal{R}$ reconnections at once) by tens of minutes. Keying it by address is rejected (#780; many honest browsers share one address behind carrier-grade NAT), and keying it by slot is void, since the node assigns slots. An honest joining storm meets none of this: every arrival is admitted at once into the cleanest slot. The levers that remain are a cost on identity, which this law does not assume, and position randomization (#780, 2.0), which takes from a fresh identity the choice of where it lands.

Charging the edge identity instead of its slot is unsound: a departing identity's floor items stay, while a fresh identity brings a fresh floor, so rotation alone grows $|T|$ past B_T.

Accounting to the origin, as the removal store did [1] and the replay table's quota does, fails part (4). An origin costs nothing, so a party controls unboundedly many, and the only bound left is the node-wide total. The slot is the unit that is both authenticated and bounded in number. The quota per origin stays: it is a rate within an edge, not a capacity.

Who overflows honestly. In an overlay of mostly browser nodes, the honest overflow is on relay edges: a browser reached through a seed sees every origin the seed forwards to it on that one edge. Without the fair claim, a party with k edges could take the whole pool and refuse every honest relay edge for the table's horizon. With it, a relay edge keeps $\lceil \mathcal{P}_T/n \rceil$ however many slots the party holds. The alternative of charging only streams whose origin is the delivering edge, and leaving relayed streams to the origin quota, reopens #907: a relayed final-destination stream is exactly the stream a free identity creates, and the quota is per origin.

IV. Replay streams (#907)

A receiver stream s admits a transaction on two paths. Live, only while its proof is live, $t \le ts + ttl$, with $ttl \le \texttt{MAX_TTL_MS} = H$ and $ts \le t + \sigma$. Out of a relay inbox, as of its hold instant h, which the witness bounds by the sender's proof, $h \le ts + ttl$; only an application message is ever held.

Definition 3 (Relay hold horizon). A relay element held at h retires at $h + H_I$, $H_I = \texttt{MAX_RELAY_INBOX_TTL_MS}$ (7 days): every replica projects it out of its carrier at a read, a join or a sync, and a drain never delivers it. The predicate reads the signed hold instant, not the element's dot, so it is element-intrinsic and commutes with join, and a member that rewrites an old hold under a fresh dot gains nothing. An element is projected out, never refused, so a carrier with an old element stays admissible. Relay removes keep their cap.

Definition 4 (Horizon). Let $x_s = \max(ts + ttl)$ over the transactions s admitted, and $\iota(\mathsf{Application}) = H_I + \sigma$, $\iota = 0$ for every other class. The stream is spent at t iff $t \ge x_s + \iota + \sigma$. So a control, storage or E2E stream of default-lifetime traffic is spent about 10 min after its last transaction, and none later than $H + 2\sigma$; an application stream $H_I + \sigma$ later.

Lemma 1 (Eviction is sound). A spent stream can be forgotten without reopening replay. Every transaction it ever admitted is then undeliverable at this node's clock, even one stepped back by σ, on either path, so a stream opened afresh at First never sees one again.

Proof. Live, a transaction is refused at its proof once $t > ts + ttl$, and $t - \sigma \ge x_s$. Out of an inbox, it is delivered only while $t < h + H_I \le ts + ttl + H_I$, and $t - \sigma \ge x_s + H_I + \sigma$. Both checks run before replay, whatever the window holds. □

Law 2 (Stream admission). The receiver table is a table under Law 1:

- A new stream is introduced by the slot of the edge that delivered its first transaction; a delivery out of the node's own relay inbox by μ; out-of-band connection signaling by λ.

- A stream leaves the table only when it is spent and holds no relay custody, by Lemma 1. Spent streams are retired lazily, by the next admission that reads the clock past them, and their records are removed in that admission's batch.
- The record of a stream stores $(\pi(s), x_s + \iota + \sigma)$, so a restart recovers the charges.

Parameters:

- $f = 32$ new streams per slot, $\mathcal{P} = 3328$, $L = 1024$ and $M = S_\mu = 768$, so $B(3) = 328 \cdot 32 + 3328 + 1024 + 768 = 15\,616$ and $B(s) = B(3) + 64(s-3)$;
- $B(s) \le C = 16\,384$, the table's hard capacity, iff $s \le 15$: up to 15 successors Theorem 1(1) holds and the floors bind first, and past it the hard capacity refuses before a floor does;
- the per-origin quota is unchanged.

A new stream past its bound is refused with TransactionReplayStreamCapacityExceeded, as today. The refusal now lasts until a stream of the refused slot is spent, at most $H + 2\sigma$ (100 min, about 10 min for default-lifetime traffic) and $H + H_I + 3\sigma$ for application streams, and it reaches only a slot past its floor and its fair claim.

Proposition 2 (No permanent lockout). A party with k slots and any number of origins can refuse new streams only through those k slots, and only once the pool is full. For every other slot, a new origin is refused once that slot holds $f + \lceil \mathcal{P}/n \rceil$ unspent streams. Every refusal ends within $H + 2\sigma$, or $H + H_I + 3\sigma$ for an application stream, of the last admission on the streams that fill the bound.

Proof. Theorem 1, and Lemma 1 for the end of a refusal. □

The application horizon is the price of offline mail: a stream must outlive every hold of its messages. It lengthens what Proposition 1 costs an honest newcomer in an application-charged slot to H_I, not what any principal can hold.

Inbox mail draws on μ, never on L, so a flood of mail cannot refuse this node's own signaling. The residual: once mail from M distinct origins is unspent, mail from a new origin is refused at delivery for up to $H_I + H + 3\sigma$; its element stays held, unless its own hold horizon passes first. A slot would not do better: a node's inbox mail all enters through its own store or one member's fetch reply, so charging the delivering slot isolates nothing.

A relay or seed that forwards many distinct origins can fill its own floor legitimately; its overflow then holds a fair claim of the pool, $\lceil \mathcal{P}/n \rceil \ge 11$ at worst and 3328 alone, and past it the honest cost is a refusal on that edge until its streams are spent. The sender table holds this node's own streams, so it is local.

V. The removal store, re-keyed

Law 6 of [1] charged each remove to its sender's origin. We re-key it under Law 1:

- an operation or a sync is charged to the slot of the edge that delivered it, and this node's own writes and repairs to λ; a read repair of a remote lookup reply is charged to the slot that delivered the reply, so lookup replies never draw on L;
- a hand-off sync of a placement this node's view makes it own, from the node its view names as the placement's previous owner (its predecessor or a successor), is charged to the custody principal χ; every other sync from a predecessor or successor is charged to its slot, as any edge's.

Parameters: $f = 128$ removes per slot, $\mathcal{P} = 16\,384$, $L = 7\,168$ and $K = S_\chi = 16\,384$, so $B(3) = 328 \cdot 128 + 16\,384 + 7\,168 + 16\,384 = 81\,920$ and $B(s) = B(3) + 256(s-3)$. A store files each item in at most 239 bytes, the widest when every record holds one remove (its key, its DID, one remove and its charge), so $B(3)$ items take at most about 19.6 MB. Theorem 1 then closes the Sybil residual that #907 was to close: one mechanism serves both tables.

A store admits one carrier per transition, at most $T_{\max} = 1024$ removes, so all-or-none binds per topic. The binding limit of a hand-off is cumulative: without χ, every topic of a range a neighbour hands over draws on that neighbour's one slot, $f + \lceil \mathcal{P}/n \rceil$ in all, and a busy range (about 10 removes a minute per topic stays ≈ 1000 unstable) would wait up to $H + \sigma$ while the new owner misses its reads. The residuals of χ: a Byzantine predecessor or successor can fill K (a non-goal, as the holder class was), and an honest range of more than K unstable removes still waits for them to stabilize.

VI. Received frames (#934)

Let lane (e, ℓ) have advertised credit A and released count R. Its liability is $\mathrm{liab} = A - R$: the frames the receiver may still have to hold, held, in flight, or credited and not yet used. Today $\mathrm{liab} \le W = 16$ per lane, but every lane holds its whole window, so $\sum \mathrm{liab}$ is bounded only by the connection count: about 1.3 GiB at $\mathcal{R}(3)$ edges.

Definition 5 (Reclaim). Every lane carries a generation g, and its credit frames name it: $\mathsf{Credit}(A, g)$. Two frames are added, each on the lane it concerns, never charged against credit, and idempotent per generation:

- $\mathsf{Shrink}(L', g)$, receiver to sender: "from generation g, your limit is L'", with $L' = \min(R + g_\ell, A)$.
- $\mathsf{Returned}(B, g)$, sender to receiver: "I committed or reserved c frames, and from now on I commit at most $B = \max(c, L')$".

The sender takes B as its limit and ignores a credit of an older generation than its own. Until the Returned arrives, the receiver counts the lane's liability at its old A and advertises nothing on it; after it, $A = B$.

Law 3 (Hard frame bound). Every lane has a floor g_ℓ and a cap W: $g_\ell \le \mathrm{liab}(e, \ell) \le W = 16$ always. The priority lane's floor is its whole window, $g_{\mathrm{prio}} = W$, and it never borrows; each of the three data lanes has $g_\ell = 1$, so the first frame of a burst on any lane never waits a round trip.

- Demand. The admission of a data lane's frame is its demand for pooled credit: a sender with queued frames

always has its floor frame to send, or a frame the receiver holds, whose admission was the demand. So no demand frame is needed.

- Pool. The receiver grants a data lane credit beyond its floor only from the node's frame pool, at most $W - g_\ell$ to one lane, and to one edge at most its fair claim of Law 1: with $p_e = \sum_\ell (\mathrm{liab}(e,\ell) - g_\ell)^+$ and $n \geq 1$ the edges that hold pooled credit or wait for it,

$$p_e \leq \lceil \mathcal{P}_F / n \rceil \text{ when lent}, \qquad \sum_e p_e \leq \mathcal{P}_F,$$
$$\sum \mathrm{liab} \leq B_F(s) = \mathcal{R}(s)\,(W+3) + \mathcal{P}_F.$$

- Demands are served first come, first served within the claims: a demand at its edge's claim waits without holding back a later one, and a lane lent more advertises it at once.
- Rebalance. While a demand of an edge holding less than its share $\lfloor \mathcal{P}_F / n \rfloor$ waits for a full pool, every edge holding more than its share is shrunk to it. The shares fit the pool, $n\lfloor \mathcal{P}_F / n \rfloor \leq \mathcal{P}_F$, as the claims need not; an edge at its share shrinks no other.
- Idle. A data lane that holds pooled credit and admitted no frame for T_{idle} is shrunk to its floor.

Parameters:

- $\mathcal{P}_F = 256$ frames (16 MiB, the soft limit it replaces);
- $T_{\mathrm{idle}} = 1$ s, on a tick the transport is driven by: the wall clock in a backend, the virtual clock in the deterministic simulation;
- $B_F(3) = 328 \cdot 19 + 256 = 6\,488$ frames, about 405 MiB at 64 KiB, and $B_F(s) = B_F(3) + 38(s-3)$ frames.

This is independent of how many connections are open.

Why $g_{\mathrm{prio}} = W$. The most priority-lane frames one edge had outstanding (held, in flight, or credited and unused) were measured over the dummy suite, with the window lifted and credit returned per frame: at most 10 under asynchronous delivery; 16 in every sync-storm case; 17 under a controlled scheduler that holds deliveries on purpose, an artificial round trip. Control above 16 already waits for credit today, and still does; no smaller floor covers the bursts. The frame count prices a control frame at the 64 KiB a data frame may take: byte-denominated credit (#937) removes that artifact, and with it most of $\mathcal{R}(s)\,W$.

The cost of a fixed T_{idle}. The transport has no round-trip estimate, so T_{idle} is a constant. On a link whose round trip exceeds 1 s, a lane that pauses for one round trip is shrunk, and its next burst waits one round trip for pooled credit again; no frame is refused.

Proposition 3 (Frames). Under Law 3, for an honest sender:

1) $\sum \mathrm{liab} \leq B_F$ and $g_\ell \leq \mathrm{liab}(e,\ell) \leq W$ in every interleaving, with credit, shrink and return frames reordered and duplicated;
2) no honest frame is refused: a frame sent under credit is within the liability counted for it;
3) a lane with queued frames is never blocked forever while the receiver keeps consuming (progress);
4) an idle lane holds at most its floor after T_{idle} plus one round trip, so it starves no active lane, and its first frame after the pause is sent at once;
5) throughput: an edge whose data lanes have frames queued holds, within one round trip of its demand, or two when the pool must be rebalanced first, at least $\min\big(3(W-1), \lfloor \mathcal{P}_F / n \rfloor\big)$ pooled frames besides its floors, $n \leq \mathcal{R}(s)$ the edges holding or waiting for pooled credit, however busy the others are;
6) control traffic never waits for the pool.

Proof sketch. (1) Pooled credit is granted only when the counted liability fits, and the counted liability never falls below the true one: a shrink takes effect only once its Returned confirms what was committed, and a credit of an older generation is ignored. (2) A frame committed under A is counted until released, and $B \geq c$. (3) A blocked lane's last admission is a pending demand. The pool refills as frames are released and idle lanes are shrunk, and service is first come, first served. (4) The shrink frees the lane's pooled credit, and its floor stays. (5) A waiting edge below its share is lent from what is free; when nothing is, every edge above its share is shrunk to it, and the frames above the shares return within one round trip, which leaves the waiting edge its share, since $\sum_{e' \neq e} \min(p_{e'}, \lfloor \mathcal{P}_F / n \rfloor) \leq (n-1)\lfloor \mathcal{P}_F / n \rfloor \leq \mathcal{P}_F - \lfloor \mathcal{P}_F / n \rfloor$. (6) The priority lane's floor is its whole window, reserved per edge. These are the model-checking obligations of the implementation, over the credit model with shrink and return frames. □

The guarantee (5) is the price of a hard bound: before #934 every lane had W frames per round trip whatever the others did. Now n busy edges share $\mathcal{P}_F$, so each keeps its whole window on all three data lanes while $n \leq 5$ ($3(W-1) = 45 \leq \lfloor 256/5 \rfloor = 51$), and the floors plus $\lfloor 256/n \rfloor$ pooled frames per round trip beyond; past 256 busy edges, the floors alone. Without the claim, six busy edges would hold the whole pool, and every other data lane would be held to its floor, one frame per round trip, for as long as they keep sending.

Why the model shrinks once. The model of Proposition 3 lets a lane be shrunk once per path, which keeps its state space finite. One is enough: the transitions read a generation only to compare it with another, so a state of generation g whose older signals have all been ignored or delivered is, renamed by $g \mapsto 0$, a state the model reaches with no shrink; and a signal of an older generation is a no-op at either end. So a second shrink starts from a state the first already covers, and one shrink with credits of the older generation still in flight, delivered, duplicated and reordered across it, covers them all.

Composition. Received frames and the mailbox are disjoint memories: a frame counts in B_F until it is handed to the mailbox, which happens only once its mailbox charge, and for a first chunk its item of O (Def. 6), is granted. So inbound memory is at most $B_F \cdot 64\,\mathrm{KiB} + M$, and neither bound consumes the other. A first chunk that waits for O holds its frame and its credit: backpressure, counted in B_F, whose pooled part the idle shrink returns.

The per-lane cap keeps Proposition 4's grace Wd_0 sound under pooled credit.

This is a wire change: a total cutover, which wire version 2 names (#1104).

VII. Reassembled messages (#932)

A message of *len* bytes travels as chunks $[i, n]$ on one lane (e, ℓ), its carrier, which is its class's lane: the sender cuts chunks of equal length d_0, the last shorter, and keeps them contiguous on the carrier. Let $r(x) = 2x$ be the retained bytes of x received bytes (the buffer and the completed copy), W the lane window (Law 3), TTL the largest time to live a chunk may carry, and

$$\rho = \frac{\text{max_message_bytes}}{TTL} = \frac{60\,\text{MB}}{100\,\text{min}} = 10\ \text{B/ms}.$$

Definition 6 (Open messages). The table O holds, in bytes, the reservation $r(n\,d_0)$ of every message whose first chunk was admitted and that has neither completed nor expired, charged to the slot of the edge that delivered that chunk (Law 1, an item being a byte), with $f_O = 256\,\text{KiB}$, $\mathcal{P}_O = 128\,\text{MiB}$, $L_O = 0$ and no dedicated share:

$$B_O(s) = \mathcal{R}(s)\,f_O + \mathcal{P}_O, \qquad B_O(3) = 210\ \text{MiB},$$

within the mailbox's $M = 256\,\text{MiB}$ for $\mathcal{R}(s) \le 512$, that is $s \le 94$. An item of O is a byte of the mailbox too: O bounds what open messages take of M, and $M - B_O$ stays for every other arrival.

Definition 7 (Credit-time). Let an open message be sampled at instants $t_0 < t_1 < \ldots$: its first chunk's admission, each later chunk's, and the receiver's sweeps, at most Δ apart. At t_k let $c_k = A - N$ be the credit on the carrier that the sender holds unused: advertised, A, and not yet used by an arrival, N. Its credit-time advances over $[t_k, t_{k+1})$ by

$$\hat{\tau} \mathrel{+}= \min\big(t_{k+1} - t_k,\ c_k\,d_0/\rho\big),$$

the time its unused credit could have taken to send at ρ. Credit the receiver holds back, by holding the frames that took it, batching its release, or not lending it pooled credit (Law 3), is not in c_k, so $\hat{\tau}$ pauses while the receiver throttles the sender. Read at the receiver, c_k also counts credit still in transit to the sender, and frames the sender already spent it on that are still in flight; neither is the sender's doing, and the rate floor's grace covers both (Prop. 4). Let b be the message's bytes received.

Law 4 (One charge per message). The first chunk $[0, n]$, of length d_0, reserves $m = r(n\,d_0)$ once, as an arrival that waits in resource order and is never refused for capacity:

- on the mailbox ledger, on the edge's budget, on the share of its carrier's class lane, and on the shared pool; and
- in O, under Law 1: within the slot's floor, or else within the pool and its fair claim, as a whole.

It is valid only while $(n-1)\,d_0 < \text{max_message_bytes}$: an honest message of at most that many bytes has $n = \lceil len/d_0 \rceil$, so $n\,d_0$ passes its length by less than one chunk.

- Later chunks must have length $\le d_0$, and they charge nothing: they wait only behind the earlier arrivals of their own stream, which is ordered per peer, lane and carrier, so behind their own message's chunks alone.
- The reassembly buffer and the completed message live inside the one reservation. At completion the message leaves O and its mailbox reservation moves with it into dispatch, on its class lane: a move, not a new charge.
- Rate floor. The message expires, attributably, once $b < \rho\,\hat{\tau} - Wd_0$. It also expires at its own $ts + ttl$. Either expiry releases the reservation.

The separate reassembly byte budget and its refusal (Rejected(Capacity)) are retired. The id and slot caps stay, as validity bounds.

Proposition 4 (No honest message refused). Under Law 4, no honest chunked message is refused or dropped for capacity, no wait can deadlock, and a sender whose carrier delivers at least ρ while it holds credit is never expired by the rate floor.

Proof. The only charge is the first chunk's, which waits as an arrival before it holds a lane ticket. The ordering laws of ResourceOrderedQueue cover it, and the capacity it waits for is released by events that do not wait on it: a later chunk waits for no capacity, and its stream holds no other message's first chunk, so no message's completion waits on capacity another message holds. The transition that used to re-charge, and could only be refused, now moves bytes already reserved.

Rate floor: over $[t_k, t_{k+1})$ the sender holds c_k unused credits and a backlog (its message is contiguous on the carrier), so it delivers $\min(c_k d_0, \rho(t_{k+1} - t_k))$ in it, which is what $\hat{\tau}$ advanced, times ρ, once its credit has reached it. Between the two, $\rho\hat{\tau}$ runs ahead of b only by the lane's outstanding liability: the frames the receiver still holds, the frames in flight to it, and the credit in transit to the sender, each counted in c_k or missing from b. That liability is at most W frames, the lane window (Sec. VI), however long the round trip. So $b \ge \rho\hat{\tau} - Wd_0$ throughout: the grace covers every outstanding credit, and a high-latency honest link is never expired. □

Proposition 5 (Open messages). Let $G = (W+1)\,d_0/\rho + \Delta$, at most 107 s plus one sweep for $d_0 \le 64\,\text{KiB}$. Under Law 4:

1) a first chunk through slot κ with $\text{use}_\kappa + m \le f_O$, and every other arrival, never waits for O, however many open messages other edges hold; open messages hold at most B_O of the mailbox;
2) an honest message that waits for the pool of O waits at most G beyond the honest messages ahead of it: an open message that sends nothing is expired within G, and a slot's next first chunk queues in resource order behind the claim already waiting;
3) past G, every open message has $\rho\,\hat{\tau} \le b + Wd_0$: the credit-time for which it holds its reservation is paid for by bytes its sender streamed.

Proof. (1) Theorem 1(2) for O; an arrival that is not a first chunk takes no item of O. (2) A sender that holds its

credit and sends nothing has $\hat{\tau}$ grow at the rate of time and $b = d_0$, so it meets $d_0 < \rho\hat{\tau} - Wd_0$ after $(W+1)d_0/\rho$, and the next sweep expires it; resource order serves the waiting claim before a later one. (3) The rate floor. □

Residual. The receiver cannot tell a sender that withholds a message from one whose scheduler is busy with higher-priority classes between chunks: both hold credit and send nothing. So a large message on a link that leaves its class less than ρ is expired. This is the cost of the rate floor; $\rho = \text{max_message_bytes}/TTL$ is the least rate at which holding a reservation is no cheaper than before #932, when it cost the bytes held. A message so expired is the sender's to send again.

VIII. Hot paths (#938)

These are performance obligations, kept apart from the security laws above.

Obligation 1 (Admission complexity).
- Each arrival, release and cancel in ResourceOrderedQueue costs $O(\log n)$ in the waiters: waiters are indexed by resource, and one ledger snapshot is taken per pass.
- Each received frame takes $O(1)$ lock acquisitions, and none of them on a node-global mutex: the node load is an atomic counter.
- The node load has hysteresis: advertisements are deferred above H_{hi} and resume below $H_{\text{lo}} = H_{\text{hi}}/2$. Deferred links are kept in a set, so a release at the limit wakes no one, and crossing H_{lo} wakes each deferred link once.
- Each peer runs one admission watcher, not one task per send.
- Unread admissions are indexed by peer.

They are verified against the sync-storm matrix, before and after each change.

IX. The reassembly barrier (#927)

The inbound actor of a connection has one lane per category $\kappa \in K = \{ctl, sto, e2e, app\}$ and a reassembly lane ρ, each serving one event at a time. Events take sequence numbers *seq* in ticket order. A reassembled message keeps its final chunk's sequence and enters the lane of its category. Before #927 a data lane waited behind every earlier unfinished reassembly, whatever its category, and the control lane waited behind none: the control exemption.

Two facts of the code refute the old argument ("at most one frame in hand-over per lane, and control is never chunked").

1) Pacing. An arrived frame is submitted awaited: its carrier's drainer hands over the next frame only once this one completed, and a final chunk's completion moves onto its reassembled message. So on the paced path each carrier has at most one event in the actor. Frames released from the pre-admission hold are not paced: up to the peer's event share, 32, enter at once, at admission. Behind such a burst a control frame without the exemption would wait for up to 32 reassembly frames.
2) Chunked control. A payload is chunked whenever it exceeds the peer's advertised max-message-size, which has no floor (#1130), so control messages carrying an SDP can arrive chunked, on the control carrier. Under the exemption, a control frame released after them in the same burst ran before their reassembled message: control reordered.

Definition 8 (Per-category barrier). The category $\kappa(e)$ of an event is its lane's for a whole frame, and its carrier's for a chunk; a chunk with no carrier, which only a test hands in, has $\kappa(e) = \top$. Let U be the reassembly events ticketed and not yet completed. A lane κ dispatches its front f iff

$$\neg\exists u \in U.\ seq(u) < seq(f) \wedge \kappa(u) \in \{\kappa, \top\}.$$

Proposition 6 (Order per category). Under Def. 8, the messages of one category are processed in sequence order, whole or chunked. A frame of category κ waits only behind unfinished reassembly of κ: at most one frame on the paced path, and at most the κ-chunks of one released burst.

Proof. A lane serves in sequence order. A reassembled message m of κ has the sequence of its final chunk u; every κ-frame f with $seq(f) > seq(u)$ waits while $u \in U$, and u leaves U in the step that inserts m into lane κ at $seq(u)$, ahead of f. The lane-mismatch refusal makes m's category its carrier's, so $\kappa(u)$ is m's lane. The bounds are pacing's and the hold's. □

Order is guaranteed per category only, and no protocol relies on cross-category order. Categories travel on separate data channels (#906), whose arrival order the transport does not relate, and the actor serves its lanes concurrently, so whole frames of two categories were never ordered; the old barrier ordered data only behind earlier reassembly. Every request and its report, and every exchange whose steps must be ordered, lies in one category:

Pair	Why order across categories is not needed
storage sync and its report (sto) against topology control (ctl)	the sync names its signed destination and is forwarded by the current topology when not ours; persistence and acks are keyed by entry
relay custody, holds and their acks against inbox fetches	all sto; relayed mail and a direct CustomMessage take different paths and were never ordered
link control against payloads	handled before the actor; keyed by delegation digest, with lease durations; an Unknown means the referencing frame was dropped and is resent
E2E handshake and stream against custom messages	the handshake and stream are all e2e; a custom message needs no E2E session
DHT control requests and reports; storage requests and reports	each pair within ctl, or within sto

Decision. Replace the node-wide barrier and its hand-off barrier by Def. 8. The control exemption is gone as a

special case: control waits only behind chunked control, which a peer causes only by advertising a limit below its control messages. A data category no longer waits behind another category's reassembly. The ablation toggle ProtectionLayer::BarrierControlExemption and its legacy feedback-loop witness are removed; the storm witnesses that a control probe behind another category's reassembly backlog is not blocked, and a regression test that a chunked control message released from the hold keeps its place among later control.

X. Summary of bounds

Table	Bound	At the bound
frames held	node $B_F(s)$; lane floor and cap W	sender waits for credit (backpressure)
mailbox bytes	edge budget, lane share, pool	arrival waits (backpressure)
open messages	$\mathcal{R}(s)f_O + \mathcal{P}_O$ by slot, fair pool, in the mailbox	first chunk waits (backpressure); expired below ρ
replay streams	$\mathcal{R}(s)f + \mathcal{P} + L + M$ by slot, fair pool	new stream refused, $\leq H + 2\sigma$ ($H + H_I + 3\sigma$ application)
removal store	$\mathcal{R}(s)f + \mathcal{P} + L + K$ by slot, fair pool	write refused, unacked
origin	rate quota within its edge	refused for the rate window
application inbox	64 MiB	dropped, counted (by design)

What a free identity consumes. An origin alone consumes nothing beyond its quota on the edge that carries it. An edge consumes its slot's floors, which outlive it until its items leave. The pool is shared, and is the most that any number of free identities can take together.

XI. Implementation order

PR	Issue	Law
1	#907	Laws 1, 2; Sec. V
2	#932	Law 4
3	#934	Law 3 (wire cutover)
4	#938	Obligation 1, measured
5	#927	Sec. IX (subtraction)
6	#1094	Def. 1; Prop. 1

PR 1 adds the slot-accounting primitive that PR 2 reuses for the mailbox, and the relay element horizon of Definition 3. PR 4 rewrites the queue that PR 2 changes, so it follows PR 2. PR 5 and PR 6 need nothing from the others.

Operator steps, in the CHANGELOG under Unreleased:

- PR 1 changes the replay record, which gains its introducer and spent instant. Remove the transaction-replay store while the node is stopped; this is the documented reset of the durable-records law. The removal store gains a slot charge, so remove dht-removals too. Held mail now expires H_I after its hold, even in a carrier that newer holds keep alive.
- PR 3 is a wire cutover with no step.

References

[1] R. J. Kung, "Removal stability: Eviction, tombstone bounds and heartbeats for the data topics of Rings," papers/removal-stability.tex, https://github.com/RingsNetwork/rings, 2026, issue #1060.

[2] ——, "Durable records: One presence and survival law for the local stores of a Rings node," papers/durable-records.tex, https://github.com/RingsNetwork/rings, 2026, session model of Issue #777, Section "Replicated custody".